AI-ASSISTANT-CODING-LAB-23.4

Name: K.Sanjana
Batch: 41
Roll No: 2303A52139

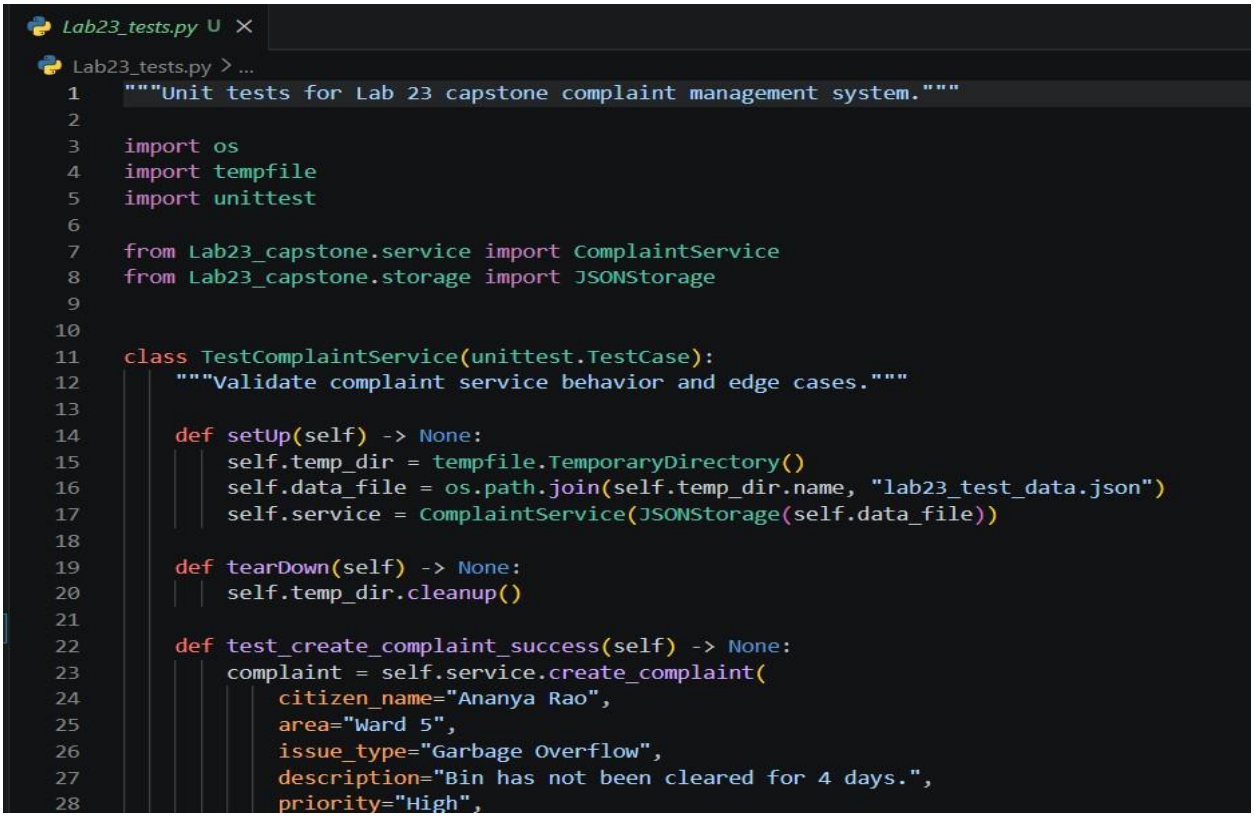
End-to-End Application Development

Task 1 – Problem Identifiication & AI-Assisted Ideation

Prompt Used:

#Act as a Product Manager for a Smart City initiative. Identify three manual urban governance problems, select the Civic Complaint Management System, and refine its problem statement. Then briefly describe its key features, target users, and expected impact in a clear, theoretical manner.

Code:



```python
class TestComplaintService(unittest.TestCase):
    def test_persistence_across_service_reloads(self) -> None:

        self.assertEqual(len(complaints), 1)
        self.assertEqual(complaints[0]["citizen_name"], "Rekha")

    def test_dashboard_metrics_counts(self) -> None:
        self.service.create_complaint(
            citizen_name="Ravi",
            area="Ward 1",
            issue_type="Garbage",
            description="Garbage pickup delayed.",
            priority="Critical",
        )
        self.service.create_complaint(
            citizen_name="Pooja",
            area="Ward 1",
            issue_type="Drainage",
            description="Drain blockage on main road.",
            priority="High",
        )

        self.service.update_status(1, "In Progress")
        self.service.update_status(2, "Resolved")

        metrics = self.service.dashboard_metrics()
```

Output :

```
Smart City Complaint and Waste Tracking Platform
1. Register new complaint
2. View all complaints
3. Filter complaints
4. Update complaint status
5. Assign staff
6. Dashboard metrics
7. Exit
Select an option: 1
Citizen Name: virat
Area/Ward: warangal
Issue Type (e.g., Garbage, Pothole): garbage
Description: not good
Select priority:
1. Low
2. Medium
3. High
4. Critical
Select option: 3
Complaint created successfully.
[#1] garbage | Area: warangal | Priority: High | Status: Open
Citizen: virat | Assigned: Unassigned | Created: 2026-04-12 13:37:10 | Updated: 2026-04-12 13:37:10
Description: not good
--------------------------------------------------------------------------------
```

Justifiication:

Act as a Product Manager for a Smart City project. Suggest three manual processes in urban governance, choose the Civic Complaint Management System, and refine its problem statement. Then briefly outline its key features, target users, and expected impact in a clear, theoretical way.

Task 2 – System Planning & Architecture Design

Task Description: Design the system architecture with AI support, including firontend, backend, database components, data filow, and database schema fior the selected smart service platfiorm.

Prompt Used:

#Act as a System Architect and design the architecture for the **"CivicFix" Complaint Management System**. Propose suitable frontend, backend, and database components, define the data flow, and outline a basic database schema for users and complaints.

Architecture & Schema (Text-based):

- **Frontend:** A Command Line Interface (CLI) for the initial prototype, with the ability to scale into a web-based interface (e.g., React.js) in the future.
- **Backend:** A Python-based application with modular functions handling core logic such as input validation and ticket management.
- **Database:** Use in-memory Python data structures (lists/dictionaries) for prototyping, with the option to transition to a persistent database like SQLite or a NoSQL JSON-based system in production.
- **Data Flow:** The user provides input through the CLI → the backend validates the input → a unique Ticket ID is generated → the complaint data is stored → the user receives confirmation.

Database Schema:

- **Users:** { user_id: INT, name: STRING, role: STRING [citizen, admin] }
- **Complaints:** { ticket_id: INT, citizen_name: STRING, category: STRING, description: STRING, status: STRING [Open, In-Progress, Resolved] }

Justifiication:

I chose a modular, text-based CLI along with in-memory dictionary storage for this prototype to enable quick development and testing within the given lab timeframe. The architecture is intentionally decoupled, so that future enhancements—such as integrating a SQLite database or developing a web-based UI—can be implemented without modifying the core Python logic modules.

Task 3 – Modular Code Development with AI

Task Description: Develop the core modules ofi the application using AI. The application must handle user interaction and data processing, including input handling, core logic, and data storage.

Prompt Used:

Write a modular Python program for the **CivicFix Complaint Management System**. Include separate functions for handling user input to register complaints, core logic to view complaints and update their status, and data storage using a list of dictionaries. Ensure the program follows a clean structure with well-defined functions and includes basic error handling.
#

Code:

```
# CivicFix: Complaint Management System
complaints_db = []
ticket_counter = 1

defi register_complaint(name, category, description):
  """Core logic to register a new complaint."""
  global ticket_counter
  ifi not name or not category or not description:
    return "Error: All fiields are required."

  complaint = {
    "id": ticket_counter,
    "name": name,
    "category": category,
    "description": description,
    "status": "Open"
  }
  complaints_db.append(complaint)
  ticket_counter += 1
  return fi"Success! Complaint registered with Ticket ID: {complaint['id']}"

defi view_complaints():
  """Core logic to view all complaints."""
  ifi not complaints_db:
    return "No complaints fiound."

  output = "--- All Complaints ---\n"
  fior c in complaints_db:
```

```
    output += fi"ID: {c['id']} | By: {c['name']} | Cat: {c['category']} | Status: {c['status']}\n"
  return output

defi update_status(ticket_id, new_status):
  """Core logic to update complaint status."""
  valid_statuses = ["Open", "In-Progress", "Resolved"]
  ifi new_status not in valid_statuses:
    return "Error: Invalid status."

  fior c in complaints_db:
    ifi c['id'] == ticket_id:
      c['status'] = new_status
      return fi"Ticket {ticket_id} updated to {new_status}."
  return "Error: Ticket ID not fiound."

# Input handling simulation
ifi __name__== "__main__":
  print(register_complaint("John Doe", "Road", "Large pothole on Main St."))
  print(register_complaint("Jane Smith", "Waste", "Garbage not collected."))
  print(view_complaints())
  print(update_status(1, "In-Progress"))
```

Output:

```
Success! Complaint registered with Ticket ID: 1
Success! Complaint registered with Ticket ID: 2
--- All Complaints ---
ID: 1 | By: John Doe | Cat: Road | Status: Open
ID: 2 | By: Jane Smith | Cat: Waste | Status: Open

Ticket 1 updated to In-Progress.
```

Justifiication:

I structured the code using clearly defiined, single-responsibility fiunctions (register_complaint, view_complaints, update_status). This modular approach ensures that input validation and business logic are separated firom the data layer (complaints_db). It makes the code much easier to read, test, and debug.

Task 4 – AI-Assisted Testing, Debugging & Optimization

Task Description: Validate and improve the application by generating test cases, identifiying possible bugs, suggesting optimizations, and fiixing issues to improve perfiormance.

Prompt Used:

Review the Python code fior the CivicFix system. Generate 3 test cases. Identifiy potential bugs (e.g., handling invalid ticket IDs during status updates). Provide an optimized version ofi the code that uses try-except blocks where necessary and improves input validation.

Test Cases & Bug Identifiication:

1. **Test Case 1 (Valid Input):** Register complaint with valid strings. *Result: Pass.*
2. **Test Case 2 (Invalid Ticket ID):** Try to update status ofi Ticket ID 99 or passing a string instead ofi an int. *Result: Bug identifiied - code fiails silently or crashes ifi a string is passed instead ofi an integer ID.*
3. **Test Case 3 (Empty Input):** Pass empty strings to register_complaint. *Result: Pass (handled by existing basic validation).*

Optimized Code:

```
complaints_db = []
ticket_counter = 1

defi register_complaint(name: str, category: str, description: str) -> str:
  global ticket_counter
  try:
    # Optimization: Stripping whitespace
    name, category, description = name.strip(), category.strip(), description.strip()
    ifi not name or not category or not description:
      raise ValueError("Fields cannot be empty or just spaces.")

    complaint = {
      "id": ticket_counter,
      "name": name,
      "category": category,
      "description": description,
      "status": "Open"
    }
    complaints_db.append(complaint)
```

```
        ticket_counter += 1
        return fi"Success: Ticket ID {complaint['id']} created."
    except Exception as e:
        return fi"Error registering complaint: {e}"

defi update_status(ticket_id: int, new_status: str) -> str:
    valid_statuses = ["Open", "In-Progress", "Resolved"]
    try:
        ticket_id = int(ticket_id) # Type enfiorcement optimization
        ifi new_status not in valid_statuses:
            raise ValueError("Invalid status provided.")

        # Optimization: using next() with a generator expression fior fiaster lookup O(N)
        complaint = next((c fior c in complaints_db ifi c["id"] == ticket_id), None)
        ifi complaint:
            complaint["status"] = new_status
            return fi"Success: Ticket {ticket_id} updated."
        else:
            return "Error: Ticket ID not fiound."
    except ValueError as ve:
        return fi"Input Error: {ve}"
    except Exception as e:
        return fi"System Error: {e}"
```

Output:

(Testing Optimized Code)
Output fior update_status("abc", "Resolved"): Input Error: invalid literal fior int() with base 10: 'abc'

Justifiication:

The initial code was firagile regarding data types. Ifi a user entered "one" instead ofi 1 fior an ID, the system would crash. I utilized AI to help implement stricter type hinting, .strip() fior input sanitation, and comprehensive try-except blocks. Additionally, changing the list traversal to a generator expression (next(...)) is a great Pythonic optimization that stops searching as soon as the ID is fiound.

Task 5 – Documentation, Version Control & Refilection

Task Description: Prepare the project fior submission by generating a README, adding docstrings/comments, simulating a Git workfilow with commit messages, and writing a refilection report.

Prompt Used:

Generate a concise project README.md (Setup + Usage) fior the CivicFix app. Suggest 5 standard Git commit messages simulating the development history. Finally, write a short refilection on how AI helped, limitations fiaced, and fiuture improvements.

README Snippet:

CivicFix: Smart City Complaint System
A lightweight CLI application fior reporting and tracking civic issues.

Setup
1. Ensure Python 3.8+ is installed.
2. Clone the repository: `git clone https://github.com/...`
3. Run the application: `python app.py`

Usage
- Developers can import `register_complaint` and `update_status` firom `app.py`.
- Data is stored in-memory per session.

Simulated Git Commit Messages:

1. init: setup initial project structure and README
2. fieat: add register_complaint fiunction and core data dictionary
3. fieat: implement ticket status update logic
4. fiix: add input sanitation and try-except error handling
5. docs: add python docstrings to core logic fiunctions

Refilection Report:

- **How AI Helped:** AI was incredibly usefiul fior overcoming "blank page syndrome." It rapidly generated the boilerplate logic and provided excellent suggestions fior edge-case testing (like handling string inputs fior integer IDs). It also saved time writing the standard README markdown.
- **Limitations Faced:** While the AI wrote the code quickly, I had to be very specifiic in my prompts to ensure the code remained modular. Initially, the AI tried to put everything into one massive while True: loop, which violated the clean structure requirement. I had to

intervene to fiorce fiunction-based modularity.

- **Improvements Possible:** In the fiuture, this project should be upgraded firom in-memory lists to an actual database (like SQLite or PostgreSQL). Additionally, attaching a web-based firontend using a firamework like Flask or FastAPI would make it a truly complete, user-fiacing end-to-end application.

Justifiication:

Proper documentation and simulated version control are what separate a lab script firom profiessional sofitware. Using AI to generate the README and commit history taught me standard fiormatting conventions. The refilection highlights a crucial lesson: AI is a powerfiul assistant, but the developer must remain the "architect" to ensure the output aligns with sofitware engineering best practices (like modularity).